Guía de ejercicios Python

Listas, diccionarios y funciones

Indicaciones generales

- Lee cuidadosamente el enunciado antes de programar.
- Define nombres de variables claros y comprensibles.
- Prueba tu programa con el ejemplo entregado y luego con al menos dos casos propios.
- No copies soluciones: explica con tus palabras qué estructura de datos o función estás usando.
- Cuando se pida una función, la función debe retornar un valor, salvo que el enunciado indique imprimir.

Objetivos de aprendizaje

Tema	Qué se practica
Listas	Crear, recorrer, filtrar, ordenar y transformar colecciones de datos.
Diccionarios	Representar datos con claves y valores, actualizar información y acumular resultados.
Funciones	Dividir problemas en bloques reutilizables con parámetros, retornos y validaciones.

Ejercicios con listas

En este bloque se practican operaciones comunes con listas: recorrido, filtrado, ordenamiento, búsqueda, modificación y creación de nuevas listas.

Ejercicio 1: Registro de notas

Objetivo: Recorrer listas y calcular estadísticas simples.

Enunciado: Crea un programa que reciba una lista de notas y muestre la nota mayor, la nota menor y el promedio.

Condiciones: No uses librerías externas. Redondea el promedio a 2 decimales.

Ejemplo de entrada/salida:

```
Entrada:
[5.5, 6.2, 4.8, 7.0, 5.9]

Salida esperada:
Mayor: 7.0
Menor: 4.8
Promedio: 5.88
```

Desafío extra: Indica cuántas notas están sobre el promedio.

Ejercicio 2: Filtrar productos baratos

Objetivo: Crear una nueva lista a partir de una condición.

Enunciado: Dada una lista de precios, genera una lista solo con los precios menores o iguales a 10000.

Condiciones: La lista original no debe modificarse.

Ejemplo de entrada/salida:

```
Entrada:
[12000, 8990, 15000, 5000, 10000]

Salida esperada:
[8990, 5000, 10000]
```

Desafío extra: Muestra también el total de productos filtrados.

Ejercicio 3: Ordenar tiempos de carrera

Objetivo: Ordenar listas numéricas y obtener posiciones.

Enunciado: Recibe una lista de tiempos en segundos y muéstralos ordenados de menor a mayor.

Condiciones: Puedes usar métodos propios de listas.

Ejemplo de entrada/salida:

```
Entrada:
[58, 72, 64, 55, 61]

Salida esperada:
[55, 58, 61, 64, 72]
```

Desafío extra: Muestra el primer, segundo y tercer lugar.

Ejercicio 4: Eliminar duplicados

Objetivo: Construir una lista sin elementos repetidos.

Enunciado: Dada una lista de nombres, crea una nueva lista manteniendo solo la primera aparición de cada nombre.

Condiciones: Mantén el orden original.

Ejemplo de entrada/salida:

```
Entrada:
["Ana", "Luis", "Ana", "Pedro", "Luis"]
```

```
Salida esperada:
["Ana", "Luis", "Pedro"]
```

Desafío extra: Cuenta cuántos elementos duplicados fueron removidos.

Ejercicio 5: Inventario bajo

Objetivo: Combinar listas relacionadas por índice.

Enunciado: Tienes dos listas: productos y cantidades. Muestra los productos cuya cantidad sea menor a 5.

Condiciones: Ambas listas tienen el mismo largo.

Ejemplo de entrada/salida:

```
Entrada:
productos = ["mouse", "teclado", "monitor", "cable"]
cantidades = [3, 8, 2, 10]

Salida esperada:
mouse
monitor
```

Desafío extra: Genera una lista con pares [producto, cantidad].

Ejercicio 6: Lista de asistencia

Objetivo: Agregar y buscar elementos en una lista.

Enunciado: Crea un programa que permita agregar estudiantes a una lista de asistencia y verificar si un estudiante asistió.

Condiciones: Evita agregar un nombre si ya existe.

Ejemplo de entrada/salida:

```
Entrada:
asistencia inicial = ["Camila", "Diego"]
agregar = "Valentina"
buscar = "Diego"

Salida esperada:
Asistencia: ["Camila", "Diego", "Valentina"]
Diego asistió: Sí
```

Desafío extra: Normaliza mayúsculas/minúsculas antes de buscar.

Ejercicio 7: Rotar una lista

Objetivo: Manipular posiciones y slicing.

Enunciado: Dada una lista, mueve el último elemento al inicio.

Condiciones: Considera qué ocurre si la lista está vacía.

Ejemplo de entrada/salida:

```
Entrada:
[10, 20, 30, 40]

Salida esperada:
[40, 10, 20, 30]
```

Desafío extra: Permite rotar la lista N veces.

Ejercicio 8: Separar pares e impares

Objetivo: Clasificar datos en dos listas.

Enunciado: Dada una lista de números enteros, crea una lista de pares y otra de impares.

Condiciones: Usa el operador módulo %.

Ejemplo de entrada/salida:

```
Entrada:
[1, 4, 7, 8, 10, 13]
```

```
Salida esperada:
Pares: [4, 8, 10]
Impares: [1, 7, 13]
```

Desafío extra: Ordena ambas listas antes de mostrarlas.

Ejercicio 9: Carrito de compras

Objetivo: Sumar elementos y aplicar condiciones.

Enunciado: Dada una lista de precios, calcula el total. Si el total supera 50000, aplica 10% de descuento.

Condiciones: Muestra los valores de forma clara.

Ejemplo de entrada/salida:

```
Entrada:
[18000, 25000, 12000]

Salida esperada:
Total sin descuento: 55000
Descuento: 5500
Total final: 49500
```

Desafío extra: Agrega una lista con nombres de productos y muestra el detalle.

Ejercicio 10: Buscar palabras largas

Objetivo: Filtrar strings según su longitud.

Enunciado: Dada una lista de palabras, muestra solo las que tienen 6 o más caracteres.

Condiciones: La longitud mínima debe guardarse en una variable.

Ejemplo de entrada/salida:

```
Entrada:
["sol", "programa", "dato", "python", "diccionario"]

Salida esperada:
["programa", "python", "diccionario"]
```

Desafío extra: Permite que el usuario defina la longitud mínima.

Bloque 2: Ejercicios con diccionarios

En este bloque se usan diccionarios para representar información estructurada, actualizar datos, contar elementos y agrupar información.

Ejercicio 11: Ficha de estudiante

Objetivo: Crear y consultar un diccionario simple.

Enunciado: Crea un diccionario con nombre, carrera y promedio. Luego muestra una frase resumen.

Condiciones: Accede a los valores usando claves.

Ejemplo de entrada/salida:

```
Entrada:
{"nombre": "Sofía", "carrera": "Informática", "promedio": 6.1}

Salida esperada:
Sofía estudia Informática y tiene promedio 6.1
```

Desafío extra: Agrega una clave "estado" según el promedio.

Ejercicio 12: Actualizar stock

Objetivo: Modificar valores dentro de un diccionario.

Enunciado: Dado un diccionario de productos y stock, actualiza la cantidad de un producto específico.

Condiciones: Si el producto no existe, muestra un mensaje.

Ejemplo de entrada/salida:

```
Entrada:
stock = {"mouse": 10, "teclado": 5}
producto = "mouse"
nueva_cantidad = 7

Salida esperada:
{"mouse": 7, "teclado": 5}
```

Desafío extra: Permite sumar o restar stock en vez de reemplazarlo.

Ejercicio 13: Contador de palabras

Objetivo: Usar diccionarios como contadores.

Enunciado: Dada una lista de palabras, cuenta cuántas veces aparece cada una.

Condiciones: No importa el orden de impresión.

Ejemplo de entrada/salida:

```
Entrada:
["python", "java", "python", "sql", "java", "python"]

Salida esperada:
{"python": 3, "java": 2, "sql": 1}
```

Desafío extra: Ignora mayúsculas/minúsculas.

Ejercicio 14: Agenda telefónica

Objetivo: Agregar, buscar y eliminar claves.

Enunciado: Crea una agenda donde el nombre sea la clave y el teléfono sea el valor. Busca un contacto.

Condiciones: Si no existe, muestra "Contacto no encontrado".

Ejemplo de entrada/salida:

```
Entrada:
agenda = {"Ana": "912345678", "Luis": "987654321"}
buscar = "Luis"
```

```
Salida esperada:
Teléfono de Luis: 987654321
```

Desafío extra: Permite eliminar un contacto.

Ejercicio 15: Notas por asignatura

Objetivo: Recorrer un diccionario y calcular promedio.

Enunciado: Dado un diccionario con asignaturas y notas, calcula el promedio general.

Condiciones: Redondea a 2 decimales.

Ejemplo de entrada/salida:

```
Entrada:
{"Matemática": 6.0, "Programación": 5.8, "Base de Datos": 6.4}

Salida esperada:
Promedio general: 6.07
```

Desafío extra: Muestra la asignatura con mayor nota.

Ejercicio 16: Agrupar estudiantes por sección

Objetivo: Usar diccionarios con listas como valores.

Enunciado: Dada una lista de pares estudiante-sección, crea un diccionario donde cada sección tenga su lista de estudiantes.

Condiciones: Crea la lista de una sección si aún no existe.

Ejemplo de entrada/salida:

```
Entrada:
[("Ana", "A"), ("Luis", "B"), ("Pedro", "A")]

Salida esperada:
{"A": ["Ana", "Pedro"], "B": ["Luis"]}
```

Desafío extra: Ordena alfabéticamente los estudiantes de cada sección.

Ejercicio 17: Catálogo de libros

Objetivo: Trabajar con diccionarios anidados.

Enunciado: Crea un catálogo donde cada código de libro apunte a un diccionario con título, autor y año.

Condiciones: No sobrescribas un código que ya exista.

Ejemplo de entrada/salida:

```
Entrada:
codigo = "L001"
datos = {"titulo": "Python Básico", "autor": "A. Pérez", "anio": 2024}

Salida esperada:
{"L001": {"titulo": "Python Básico", "autor": "A. Pérez", "anio": 2024}}
```

Desafío extra: Busca libros por autor.

Ejercicio 18: Votación simple

Objetivo: Contabilizar votos usando claves.

Enunciado: Dada una lista de votos, cuenta cuántos votos obtuvo cada opción.

Condiciones: Muestra también la opción ganadora.

Ejemplo de entrada/salida:

```
Entrada:
["A", "B", "A", "C", "B", "A"]
```

```
Salida esperada:
{"A": 3, "B": 2, "C": 1}
```

Desafío extra: Considera el caso de empate.

Ejercicio 19: Conversor de moneda fijo

Objetivo: Consultar valores en un diccionario.

Enunciado: Crea un diccionario con tasas de conversión fijas desde CLP a otras monedas y calcula una conversión.

Condiciones: Usa tasas entregadas en el ejercicio, no consultes internet.

Ejemplo de entrada/salida:

```
Entrada:
tasas = {"USD": 0.0011, "EUR": 0.0010}
monto_clp = 10000
moneda = "USD"

Salida esperada:
10000 CLP equivalen a 11.0 USD
```

Desafío extra: Permite convertir a todas las monedas disponibles.

Ejercicio 20: Resumen de ventas por vendedor

Objetivo: Acumular montos por clave.

Enunciado: Dada una lista de ventas con vendedor y monto, calcula el total vendido por cada vendedor.

Condiciones: Si un vendedor aparece varias veces, acumula sus ventas.

Ejemplo de entrada/salida:

```
Entrada:
[("Ana", 15000), ("Luis", 20000), ("Ana", 10000)]

Salida esperada:
{"Ana": 25000, "Luis": 20000}
```

Desafío extra: Muestra el vendedor con mayor total.

Bloque 3: Ejercicios con funciones

En este bloque se ejercita la creación de funciones con parámetros, valores de retorno, validaciones y reutilización de lógica.

Ejercicio 21: Calcular promedio

Objetivo: Definir una función con parámetros y retorno.

Enunciado: Crea una función calcular_promedio(notas) que reciba una lista de notas y retorne el promedio.

Condiciones: La función no debe imprimir; debe retornar el valor.

Ejemplo de entrada/salida:

```
Entrada:
calcular_promedio([5.0, 6.0, 7.0])

Salida esperada:
6.0
```

Desafío extra: Controla el caso de lista vacía.

Ejercicio 22: Validar edad

Objetivo: Usar condicionales dentro de una función.

Enunciado: Crea una función es_mayor_edad(edad) que retorne True si la edad es mayor o igual a 18.

Condiciones: Usa valores booleanos, no textos.

Ejemplo de entrada/salida:

```
Entrada:
es_mayor_edad(20)

Salida esperada:
True
```

Desafío extra: Valida que la edad no sea negativa.

Ejercicio 23: Saludo personalizado

Objetivo: Crear funciones que retornan strings.

Enunciado: Crea una función saludar(nombre, carrera) que retorne un saludo personalizado.

Condiciones: El texto debe respetar el formato indicado.

Ejemplo de entrada/salida:

```
Entrada:
saludar("Javiera", "Informática")

Salida esperada:
Hola Javiera, bienvenida a Informática.
```

Desafío extra: Agrega un parámetro opcional para la jornada.

Ejercicio 24: Aplicar descuento

Objetivo: Diseñar funciones reutilizables para cálculos.

Enunciado: Crea una función aplicar_descuento(precio, porcentaje) que retorne el precio final.

Condiciones: El porcentaje se recibe como número entero o decimal.

Ejemplo de entrada/salida:

```
Entrada:
aplicar_descuento(20000, 15)

Salida esperada:
17000.0
```

Desafío extra: Evita descuentos menores a 0 o mayores a 100.

Ejercicio 25: Contar vocales

Objetivo: Recorrer strings dentro de una función.

Enunciado: Crea una función contar_vocales(texto) que retorne cuántas vocales contiene el texto.

Condiciones: Considera vocales en mayúscula y minúscula.

Ejemplo de entrada/salida:

```
Entrada:
contar_vocales("Python es entretenido")

Salida esperada:
7
```

Desafío extra: Incluye vocales con tilde.

Ejercicio 26: Normalizar nombres

Objetivo: Transformar listas usando funciones.

Enunciado: Crea una función normalizar_nombres(nombres) que reciba una lista y retorne los nombres con primera letra mayúscula.

Condiciones: La función debe retornar una nueva lista.

Ejemplo de entrada/salida:

```
Entrada:
normalizar_nombres(["ana", "LUIS", "pEdRo"])

Salida esperada:
["Ana", "Luis", "Pedro"]
```

Desafío extra: Elimina espacios al inicio y al final.

Ejercicio 27: Buscar máximo

Objetivo: Implementar lógica sin depender directamente de max().

Enunciado: Crea una función buscar_maximo(numeros) que retorne el número mayor de una lista.

Condiciones: No uses max() en la primera versión.

Ejemplo de entrada/salida:

```
Entrada:
buscar_maximo([4, 9, 2, 15, 6])

Salida esperada:
15
```

Desafío extra: Retorna None si la lista está vacía.

Ejercicio 28: Generar reporte de estudiante

Objetivo: Combinar diccionarios y funciones.

Enunciado: Crea una función reporte_estudiante(estudiante) que reciba un diccionario y retorne un texto resumen.

Condiciones: Asume que existen las claves nombre y promedio.

Ejemplo de entrada/salida:

```
Entrada:
reporte_estudiante({"nombre": "Tomás", "promedio": 5.7})

Salida esperada:
Tomás tiene promedio 5.7.
```

Desafío extra: Agrega estado: aprobado si promedio >= 4.0.

Ejercicio 29: Calcular IVA

Objetivo: Crear funciones pequeñas y reutilizables.

Enunciado: Crea una función calcular_iva(monto) que retorne el IVA correspondiente al 19%.

Condiciones: No imprimas dentro de la función.

Ejemplo de entrada/salida:

```
Entrada:
calcular_iva(10000)

Salida esperada:
1900.0
```

Desafío extra: Crea otra función total_con_iva(monto).

Ejercicio 30: Menú de operaciones

Objetivo: Usar funciones para organizar un programa.

Enunciado: Crea funciones para sumar, restar, multiplicar y dividir. Luego usa una variable operación para decidir cuál ejecutar.

Condiciones: Controla la división por cero.

Ejemplo de entrada/salida:

```
Entrada:
a = 12
b = 4
operacion = "dividir"

Salida esperada:
Resultado: 3.0
```

Desafío extra: Permite que operación acepte símbolos: +, -, *, /.